QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Content

Document Control **3**
PRELIMINARY ROUND (Test Strategy & Planning) **4**
1. Scope & Requirements Traceability 4
2. Risk Assessment & Mitigation Strategy 5
3. Test Environment & Execution Strategy 7
4. CI/CD Release Thresholds & Automation Gates 9
4.1 Integration Thresholds (Merging to Main) 9
4.2 Deployment Thresholds (Pushing to Production) 10
5. Test Case Specifications 11
6. AI Output & Boundary Testing 13
6.1. Prompt/Response Test Pairs 13
6.2. Oversized/Larger Input Test 15
6.3. Adversarial/Edge Prompt Test 15
6.4. Hallucinating Handling 15

© UMHackathon 2026

Email: umhackathon@um.edu.my

Document Control

Field	Detail
System Under Test (SUT)	UMHackathon 2026 - Team **Bug Hunters**
Team Repo URL	*https://github.com/Empty-0/UM-Hackathon-2026---Intelligent-Supply-Chain-Agent*
Project Board URL	*https://github.com/users/TANGYUELI/projects/2*
Live Deployment URL (optional for preliminary round)	-

Objective:

The primary objective is to ensure that the Intelligent Cross-Border Supply Chain Agent reliable process unstructured inputs such as WhatsApp messages and PDF invoices, transform them into structured procurement requests and execute end-to-end supply chain decision-making through multi-agent coordination, including the ingestion agent, sourcing agent and execution agent. This helps to maintain system stability, concurrency of traffic and CI/CD checkpoints.

© UMHackathon 2026

Email: umhackathon@um.edu.my

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

This section defines the testing scope for the Intelligent Cross-Border Supply Chain Agent and maps all test cases to system requirements via a Requirement Traceability Matrix (RTM). It ensures all multi-agent functions, including ingestion, sourcing, and execution. Besides that, it also prevents system failures and unplanned feature deviations.

In-Scope Core Features

- **Multi-Language Input Processing:** Understand and process procurement requests from multilingual text inputs
- **Procurement Requirement Extraction:** Extract product name, quantity, halal requirement, maximum cost and lead time
- **Supplier Searching & Comparison:** Search and generate a list of potential suppliers based on user requirements
- **Risk Detection:** Identify issues such as seasonal shipping disruptions, long-haul routing issues and delivery delays
- **Decision Recommendation:** Generate optimal supplier recommendations based on cost, lead time and risk evaluation

Out-of-Scope

- **Payment Processing:** No support for payment transactions
- **Real-Time Logistics Tracking:** No live tracking of shipment status or delivery location
- **User Account Management:** No user authentication, profiles or account-based history tracking
- **Automated Supplier Communication:** System does not send emails or place orders automatically

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. Risk Assessment & Mitigation Strategy

This section identifies potential technical risks associated with the Intelligent Cross-Border Supply Chain Agent, particularly within its multi-agent architecture, unstructured data processing, and automated decision-making workflow. Each risk is evaluated using a 5×5 Risk Assessment Matrix, where:

Technical Risk	**Likelihood (1–5)**	**Severity (1–5)**	**Risk Score (L×S)**	**Mitigation Strategy**	**Testing Approach**
Incorrect extraction of procurement details such as wrong quantity or product from multilingual input	3	5	15 (High)	Implement structured JSON schema validation and re-prompting mechanism for failed or low-confidence extraction	Provide multilingual and noisy inputs such as mixed language WhatsApp text and verify extracted fields accuracy
Selection of invalid supplier due to incomplete data	3	4	12 (High)	Apply constraint-based filtering based on cost, lead time, availability, and supplier ranking thresholds	Simulate multiple supplier datasets with conflicting attributes and verify correct ranking output
Failure to detect logistics risks	2	5	10 (Medium)	Integrate predefined risk rules and anomaly detection checks for logistics conditions	Inject simulated disruption scenarios and verify if system flags risk correctly

© UMHackathon 2026

Email: umhackathon@um.edu.my

Incorrect supplier recommendation due to inaccurate data extraction or flawed AI reasoning	3	5	15 (High)	Add validation checks, use ranking logic with weighted criteria and provide justification for supplier recommendation	Validate recommendation output against expected results based on predefined rules and perform edge case testing

Risk Assessment Scoring Criteria:

Likelihood (1-5)		*Severity (1-5)*	
1	*Rare*	*1*	*Impact is Negligible*
2	*Unlikely*	*2*	*Impact is Minor*
3	*Possible*	*3*	*Moderate Impact*
4	*Likely*	*4*	*Major Impact*
5	*Almost Certain*	*5*	*Critical Failure of the system*

Risk Score = Likelihood × Severity

Risk Level Reference:

Risk Score	***Risk Level***	***Recommended Action***
1 – 5	*Low*	*Monitor only. Acceptable risks.*
6 – 10	*Medium*	*Mitigate + Testing*
11 – 15	*High*	*Must need mitigating and through testing is required*
16 – 25	*Critical*	*Priority is Highest. Need extensive level of testing.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

3. Test Environment & Execution Strategy

Unit Test

Scope: Ingestion Agent, Sourcing Agent, Risk Detection rules.
Execution: Tests are implemented using frameworks such as PyTest and executed during development and automatically in the CI pipeline upon code push.
Isolation: External dependencies such as supplier databases and logistics data are mocked to focus on core logic such as extraction accuracy and ranking algorithm.
Pass Condition:

- Correct extraction of structured fields like product, quantity, budget and other constraints.
- Correct supplier ranking based on predefined logic.
- Proper handling of edge cases such as missing input and multilingual text.

Integration Test

Scope: End-to-end workflow across ingestion agent, sourcing agent, execution agent.
Execution: Performed after unit testing, especially when integrating multiple agents.
Workflow: Real data flow is tested without mocking to ensure correct interaction between agents, including state passing in LangGraph.
Pass Condition:

- Correct data flow between agents.
- Accurate supplier recommendation based on input.
- System produces consistent and expected decision output.

Test Environment (CI/CD Practice):

Local Testing: Manual testing using local environment with sample inputs,
Staging/CI: Automated testing triggered via GitHub Actions on every code push.
CI/CD Pipeline:

- Unit tests executed at feature branch level.
- Integration tests executed during pull request to main branch.
- Ensures system stability before deployment.

© UMHackathon 2026

Email: umhackathon@um.edu.my

Regression Testing & Pass/Fail Rules:

Execution Phase: Regression testing is performed after each update to ensure all core features remain functional.

Pass/Fail Condition:A test case passes only when actual output matches expected output or meets defined thresholds. Any incorrect extraction or wrong recommendation is logged as a defect.

Continuation Rule: Testing progresses only when critical modules meet defined thresholds. Failed critical tests must be resolved before proceeding.

Test Data Strategy:

Manual:

- Create diverse test inputs including multilingual text, incomplete requests, and edge-case scenarios
- Simulate real-world user requests such as halal requirements, budget constraints and lead time limits.

Automated:

- Use predefined JSON datasets and mock supplier data for automated testing.
- Generate synthetic data for varying supplier attributes (price, lead time, availability, risk conditions).

Passing Rate Threshold:

A minimum of 86.2% of all test cases that are executed must pass. For the critical tests, all the tests (100%) need to be passed and there is no exception for that. Because critical failure costs huge in production.

© UMHackathon 2026

Email: umhackathon@um.edu.my

4. CI/CD Release Thresholds & Automation Gates

This section is mainly for defining metrics & threshold for success criteria. So, if a certain threshold is not met, the pipeline blocks the forward transition.

4.1 Integration Thresholds (Merging to Main)

Checks	Requirements	Project	Pass/Failed
Automatic Build	**Zero Build Error**	2 Errors	Failed
Unit Tests	**100% Passing Rate**	100%	Passed
Code Quality	**Zero Linting Error**	100%	Passed
Test Coverage	**Minimum 81.2%**	93.5%	Passed

© UMHackathon 2026

Email: umhackathon@um.edu.my

4.2 Deployment Thresholds (Pushing to Production)

Checks	Requirements	Project	Pass/Failed
Regression Test	**Minimum 90%**	100%	Pass
AI Output Pass Rate	**Minimum of 80% of documented prompt**	100%	Pass
Critical Bugs	**Zero P0/01 Bugs**	None	Pass
API Performance	**Response Time < 800ms**	Average acceptance response time: 45ms, 0% error rate	Pass
Security	**API keys are not exposed**	Clean	Pass

© UMHackathon 2026

Email: umhackathon@um.edu.my

5. Test Case Specifications

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-01	**Happy Case (Entire Flow):** End-to-end Workflow	Test full system flow from unstructured input to final supplier recommendation	Input message: "I need 100 earbuds, budget RM2000, delivery within 5 days" System extracts requirements System searches suppliers based on requirement System ranks and recommends best supplier	System correctly extracts product, quantity, halal requirement, budget, lead time and returns recommended supplier	The system processed the natural language input successfully. Agent 1 extracted the intent into a JSON object (Product: Earbuds, Qty: 100, Budget: 2000, Delivery: 5 days). Status: Passed
TC-02	**Specific Case (Negative):** No Match	System gracefully handles product that has no matching supplier	Input: "Need 100 units of Quantum Supercomputer"	Agent 2 returns "status": "no_match" for that item, pipeline continues without crashing, and Agent 3 generates a general inquiry email	System returned no_match, no exception, fallback email generated Status: Passes

© UMHackathon 2026

Email: umhackathon@um.edu.my

TC-03	***NFR (Performance):*** Concurrent Requests	Evaluate system performance under low concurrent load (5 simultaneous requests) using asynchronous processing (immediate 202 accepted + background task)	1. Send 5 procurement requests concurrently to /run-async endpoint 2. Measure response time for the initial 202 Accepted 3. Poll for final results	Average response time for acceptance < 100ms, 0% error rate, final results eventually delivered	Average acceptance response time: 45ms, 0% error rate, all requests completed within 3s Status: Pass
TC-04	***NFR (Load):*** Supplier Sourcing Efficiency	Test system efficiency when handling large supplier dataset	Provide large supplier dataset such as 1000 set or above Run sourcing and ranking process	Average response time < 800ms in addition 0% error rate.	*Average response time is 85ms (1 item) to 210ms (3 items), error rate is 0%, no failures in matching, parsing or ranking.* *Status: Passes*

© UMHackathon 2026

Email: umhackathon@um.edu.my

6. AI Output & Boundary Testing

This section is designed to validate that your GLM integration does produce acceptable output and handles any inputs that are abnormal gracefully.

6.1. Prompt/Response Test Pairs

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	Need 100 earbuds, budget RM2000, deliver to KL within 5 days	Extracts product = earbuds, quantity = 100, halal requirement= null, budget = RM2000, lead time = 5 days and returns structured JSON with recommended supplier	Correctly extracted product= earbud, qty= 100, max_price= 2000, lead_time_days= 5. Return recommended supplier.	Pass
AI-02	Looking to buy 40 units of NeoCam 4K Action Camera. The budget is RM300 each.	Extracts product="NeoCam 4K Action Camera", qty=40, max_price=300. Agent 2 evaluates suppliers (RM229.97, RM200.97, RM20.00). The RM20.00 supplier is flagged because it falls below the 50% average price threshold.	Extracted entities correctly. Agent 2 flagged the RM20.00 supplier with "Abnormally low price - Potential Fraud/Quality Risk" and selected the RM200.97 supplier as the safe winner.	Pass

© UMHackathon 2026

Email: umhackathon@um.edu.my

AI-03	要100个耳机 pls budget rm2000 urgent need fast delivery KL	Correctly processes mixed language, which is a mix of Chinese, English and Malay. System extracts all key entities and normalizes into structured format	Agent 1 correctly extracted: 耳机 (headphones), 100 units, rm2000. Agent 2: no_match. Agent 3: RFI email generated. The system handled mixed language.	PASS

© UMHackathon 2026

Email: umhackathon@um.edu.my

6.2. Oversized/Larger Input Test

Fields	Details
Maximum Input Size	10 pages uploaded
Input used while testing	60 pages uploaded - 6x of the limit
Expected Behavior	System reject
Actual Behavior	Request failed
Status	Pass

6.3. Adversarial/Edge Prompt Test

Prompt Input	GLM response	System Handling
cheap fast need urgent buy everything send now	Fails to identify key features such as product name, product category, quantity and budget constraints. Unreliable supplier recommendations are provided due to insufficient structured information.	System flags incomplete request. Workflow is stopped before Sourcing Agent execution

6.4. Hallucinating Handling

For our system, there are multiple methods to detect hallucinating responses.
Structured output validation:
Ensure all responses follow predefined JSON schema, which include product name, product category, quantity, budget and lead time.
Source Consistency Check:
Verified supplier data is available in the supplier data set.

© UMHackathon 2026

Email: umhackathon@um.edu.my

If the hallucinating responses are detected, our system will mitigate it.

- Validated and structured output only can be passed to the next step.
- System blocks or rejects supplier recommendations that cannot be verified.

© UMHackathon 2026

Email: umhackathon@um.edu.my